

KVM-Based eBPF Rootkit Detection Engine & Toolkit

Test Document

Members:

Luis Abraham - labrahamesco2024@my.fit.edu

Dominick Morales - dmorales2024@my.fit.edu

Braiden Ames - bames2024@my.fit.edu

Dylin Irons - diron2024@my.fit.edu

Advisor:

Dr. Ribeiro - eribeiro@fit.edu

Client:

Amy Alvarez - NASA Scientist

Florida Institute of Technology Senior Design

Table of Contents

- 1. Introduction**
 - 1.1. Purpose
 - 1.2. Scope
 - 1.3. Definitions, Acronyms, and Abbreviations
 - 1.4. References
- 2. Test Environment and Tools**
 - 2.1. Hardware and Software
 - 2.2. Testing Tools
- 3. Functional Testing**
 - 3.1. VM Management
 - 3.2. x86-64 VMI
 - 3.3. ARM64 pKVM
 - 3.4. Linux Kernel Introspection
 - 3.5. eBPF Introspection and Telemetry
 - 3.6. VM Exit Monitoring
- 4. Toolkit and Web Interface Testing**
 - 4.1. Python Toolkit
 - 4.2. Web Interface
- 5. Integration Testing**

1. Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements of RootView, a virtual machine introspection platform designed to support the observation and eventual detection of malicious activity within the Linux kernel, with an emphasis on eBPF-based malware and rootkits.

The document establishes the expected capabilities, interfaces, constraints, and performance characteristics of the RootView system. It will serve as a reference for implementation, testing, and evaluation throughout the senior design project.

1.2 Scope

RootView is a research platform that uses virtualization-based introspection to observe Linux guest systems from outside the guest operating system. The platform aims to provide researchers with tools for examining guest memory, processor state, kernel structures, processes, and eBPF activity without relying exclusively on software executing within the monitored guest.

The system will support multiple virtualization architectures. On x86-64 systems, RootView will utilize KVM, QEMU, and KVMi to access and inspect guest state. On ARM64 systems, RootView will investigate and implement pKVM extensions that enable virtual machine introspection through the EL2 hypervisor environment.

RootView will also incorporate VM exit hooks to observe selected events occurring during guest execution. These observations will supplement traditional VMI capabilities and provide additional information about guest behavior.

The initial system will focus on establishing reliable VM management, introspection, and observation capabilities. Future development will expand these capabilities into a reusable research API and an eBPF malware detection engine that correlates hypervisor-level observations with guest kernel telemetry.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
VMI	Virtual Machine Introspection
KVM	Kernel-based Virtual Machine
KVMi	KVM Virtual Machine Introspection interface
pKVM	Protected KVM
EL2	ARM Exception Level 2, associated with hypervisor execution

eBPF	Extended Berkeley Packet Filter
BTF	BPF Type Format
DWARF	Debugging information format used to describe program and data structures
VM exit	A transition from guest execution to hypervisor execution
VM exit hook	A mechanism for observing or handling selected VM exit events
Guest	The virtual machine being monitored
Hypervisor	The virtualization layer responsible for managing guest execution
Kernel profile	Information used to interpret Linux kernel structures and symbols

1.4 References

- KVM documentation and implementation.
- LibVMI documentation.
- Linux KVM documentation.
- Linux pKVM documentation and relevant ARM virtualization documentation.
- Linux eBPF and BTF documentation.
- RootView project plan and design document.

2. Test Environment and Tools

2.1 Hardware and Software

The following environments will be used where applicable:

Component	Test Environment
x86-64 host	Linux with KVM and QEMU
x86-64 introspection	KVM and LibVMI, where applicable
ARM64 host	Compatible ARM64 system supporting pKVM
ARM64 introspection	pKVM userspace, EL1, and EL2 components

Guest OS	Supported Ubuntu/Linux distributions
Backend	C/C++
Toolkit	Python
Web interface	Supported web browser and backend service

2.2 Testing Tools

Testing may utilize QEMU, GDB, Linux debugging tools, eBPF utilities, kernel logs, and custom test programs.

3. Functional Testing

3.1 VM Management

Test ID	Test Description	Expected Result
VM-001	Create a VM using a valid configuration.	VM is created with the specified configuration.
VM-002	Start and stop a configured VM.	VM transitions to the requested state.
VM-003	Connect RootView to a running VM.	The correct VM is identified and accessed.
VM-004	Attempt to connect to a nonexistent VM.	RootView reports an appropriate error.
VM-005	Perform introspection while the guest is running.	The guest remains operational during supported operations.

3.2 x86-64 VMI

Test ID	Test Description	Expected Result
VMI-001	Read a known guest physical memory address.	Returned data matches the known memory contents.
VMI-002	Read a known guest virtual memory address.	The correct memory contents are returned.

VMI-003	Translate a virtual address to a physical address.	Translation matches the expected mapping.
VMI-004	Retrieve guest processor registers.	Returned register values match the expected guest state.
VMI-005	Read memory across a page boundary.	Data is returned correctly across both pages.
VMI-006	Request an invalid or inaccessible address.	The operation fails safely and reports an error.
VMI-007	Read memory repeatedly while the guest is active.	Operations complete without unintended guest disruption.

3.3 ARM64 pKVM

These tests validate the complete userspace-to-EL2 communication path.

Test ID	Test Description	Expected Result
ARM-001	Initialize the pKVM userspace module.	The module initializes and establishes communication with EL1.
ARM-002	Issue a valid IOCTL request.	The EL1 module receives and processes the request.
ARM-003	Issue a supported introspection hypercall.	The EL2 module processes the request and returns a response.
ARM-004	Read known guest physical memory through pKVM.	Returned data matches the expected memory contents.
ARM-005	Retrieve supported guest processor state.	The returned state matches the expected values.
ARM-006	Submit an invalid or malformed request.	The request is rejected without corrupting guest or hypervisor state.
ARM-007	Attempt an unsupported introspection operation.	The system reports that the operation is unsupported.

ARM-008	Perform introspection while the guest is running.	The guest continues executing as expected.
ARM-009	Verify access restrictions around protected guest memory.	The implementation does not permit unauthorized access.
ARM-010	Run the pKVM module against the emulated EL2 environment used for development.	Module initializes and behaves consistently with the design, validating logic before hardware testing.
ARM-011	Deploy and load the custom pKVM kernel module on an unlocked Pixel 7.	Module loads successfully and establishes EL1/EL2 communication on real hardware.
ARM-012	Compare introspection results (memory read, register state) between the emulated environment and the real Pixel 7.	Results match between emulation and hardware, confirming the emulator is a valid development stand-in.
ARM-013	Verify the module cannot be loaded on a locked/production bootloader.	The module fails to load or is rejected, confirming it does not function outside unlocked research devices.
ARM-014	Document and verify that use of the module bypasses pKVM's intended isolation guarantees.	Behavior is confirmed and flagged as research-only; no claim of safety on non-research devices.

3.4 Linux Kernel Introspection

Test ID	Test Description	Expected Result
LINUX-001	Load the appropriate kernel profile.	RootView successfully loads the required metadata.
LINUX-002	Resolve a known kernel symbol.	The symbol resolves to the expected address.
LINUX-003	Interpret a known kernel structure.	Structure fields match trusted reference information.

LINUX-004	Enumerate running guest processes.	RootView identifies the expected processes.
LINUX-005	Inspect kernel page tables.	Address translation matches the expected mappings.
LINUX-006	Attempt introspection with unavailable metadata.	RootView reports the missing metadata or unsupported operation.
LINUX-007	Test against different supported kernel versions.	Supported structures are interpreted correctly for each version.

3.5 eBPF Introspection and Telemetry

Testing will use controlled eBPF programs and maps created specifically for validation.

Test ID	Test Description	Expected Result
BPF-001	Load a known eBPF program into the guest.	RootView identifies the program.
BPF-002	Inspect program metadata.	Retrieved metadata matches the known program.
BPF-003	Create and inspect a BPF map.	RootView identifies the map and its supported metadata.
BPF-004	Attach a program to a supported hook.	RootView identifies the expected attachment.
BPF-005	Generate a supported eBPF event.	The event collector captures the event.
BPF-006	Inspect the standardized event output.	The event contains the expected fields and values.
BPF-007	Remove a test program or attachment.	RootView reflects the updated guest state.

3.6 VM Exit Monitoring

Test ID	Test Description	Expected Result
---------	------------------	-----------------

EXIT-001	Trigger a supported VM exit event.	The corresponding hook is invoked.
EXIT-002	Inspect the captured event information.	The expected event metadata is available.
EXIT-003	Generate multiple supported events.	Events are processed without unintended loss or corruption.
EXIT-004	Monitor a guest while it executes a workload.	The guest continues operating as expected.
EXIT-005	Disable monitoring, where supported.	Disabled events are no longer collected through the configured hook.
EXIT-006	Test event delivery to the RootView backend.	Events are received and represented correctly.

4. Toolkit and Web Interface Testing

4.1 Python Toolkit

Test ID	Test Description	Expected Result
API-001	Initialize the toolkit with a supported backend.	The toolkit connects successfully.
API-002	Request guest memory through Python.	Correct memory data is returned.
API-003	Request Linux process information.	Structured process information is returned.
API-004	Request eBPF program information.	Expected program metadata is returned.
API-005	Perform an unsupported operation.	A consistent error is returned.
API-006	Execute repeated API operations.	The toolkit remains operational without resource leaks or crashes.

4.2 Web Interface

Test ID	Test Description	Expected Result
WEB-001	Open the web interface.	The interface loads successfully.
WEB-002	View configured virtual machines.	VM information is displayed correctly.
WEB-003	Select a VM and request introspection.	The backend processes the request and returns results.
WEB-004	View Linux and eBPF information.	Results are displayed in a readable format.
WEB-005	View VM exit events.	Captured events appear with relevant information.
WEB-006	Trigger a backend error.	The interface displays an appropriate error message.
WEB-007	View detection results, when implemented.	Results and supporting information are displayed correctly.

5. Integration Testing

Integration tests will verify that individual components operate correctly when connected.

Test ID	Test Description	Expected Result
INT-001	Perform a memory read through the complete x86-64 backend.	The request reaches KVM and returns correct data to RootView.
INT-002	Perform a memory read through the complete ARM64 pKVM stack.	The request reaches EL2 and returns correct data to RootView.
INT-003	Use the same high-level memory interface with both backends.	Both backends provide the expected common functionality.

INT-004	Enumerate Linux processes through the VMI abstraction.	Correct process information is returned through the selected backend.
INT-005	Collect VM exit events and expose them through the toolkit.	Events reach the appropriate higher-level component.
INT-006	Correlate controlled eBPF activity with VMI observations.	The system identifies the expected relationship between observations.
INT-007	Request introspection through the web interface.	The request reaches the backend and results are returned to the UI.